In [ ]:
```
import marimo as mo
```

TP2 — Exercício 1: Game of Life modificado

Lógica Computacional · 2026/2027

1. Formulação do problema

Modelamos uma FSM (FOTS) $\Sigma = (X, I, T)$ numa grelha de dimensão $N \times N$, com $N > 16$.

O estado contém:

- **Células normais** $(1 \ldots N, 1 \ldots N)$: variáveis booleanas. O bloco inicial 3×3 está vivo e as restantes células estão mortas.
- **Fronteira inicial** ($i = 0$ ou $j = 0$): $2N + 1$ células, cada uma inicialmente viva com probabilidade $1/2$, permanecendo constante durante a evolução.
- **Fronteira final** ($i = N + 1$ ou $j = N + 1$): $2N + 3$ células permanentemente mortas, representadas por constantes booleanas.

A evolução utiliza a vizinhança de Moore, composta pelos oito vizinhos de cada célula, e a regra B3/S23:

- Uma célula morta nasce com exatamente três vizinhos vivos.
- Uma célula viva sobrevive com dois ou três vizinhos vivos.
- Nos restantes casos, a célula fica morta.

Relações de transição

Simultânea: todas as células normais aplicam a regra ao mesmo estado anterior.

Individual: exatamente uma célula normal aplica a regra por transição, podendo conservar o seu valor. Esta interpretação distingue atualização de mudança de estado.

Um seletor auxiliar inteiro q identifica a célula escolhida no modo individual. Cada transição tem um seletor independente.

Tratamento das fronteiras

A fronteira inicial pode ser:

- **Fixa:** gerada com uma semente pseudoaleatória, permitindo reproduzir a configuração.

- **Simbólica:** os valores iniciais são livres, permitindo considerar todas as atribuições possíveis.

A fronteira simbólica não representa uma distribuição de probabilidade. A fronteira final permanece morta em ambas as modalidades.

Referências: enunciado do TP2, regras gerais dos trabalhos práticos, capítulos 3 e 4 e fichas práticas 7, 8 e 9 da disciplina.

In [ ]:
```python
# Modelação da FSM desenvolvida com assistência do ChatGPT.
import random
from pysmt.shortcuts import And, Bool, Equals, Iff, Int, Ite, Not, Or, Plus,
Symbol, is_sat
from pysmt.typing import BOOL, INT

def declare(t, N):
    if N < 3:
        raise ValueError("N >= 3 nos testes; N > 16 no exercício")
    return {(i, j): Symbol(f"x_{i}_{j}_{t}", BOOL)
            for i in range(N + 1) for j in range(N + 1)}

def fronteira_aleatoria(N, semente):
    rng = random.Random(semente)
    return {(i, j): rng.getrandbits(1)
            for i in range(N + 1) for j in range(N + 1) if i == 0 or j == 0}

def init(X, N, fronteira=None):
    formulas = [X[i, j] if i <= 3 and j <= 3 else Not(X[i, j])
                for i in range(1, N + 1) for j in range(1, N + 1)]
    if fronteira is not None:
        pontos = {(i, j) for i in range(N + 1) for j in range(N + 1)
                  if i == 0 or j == 0}
        if set(fronteira) != pontos or any(v not in (0, 1) for v in
fronteira.values()):
            raise ValueError("Fronteira inválida")
        formulas += [X[p] if fronteira[p] else Not(X[p]) for p in sorted(pontos)]
    return And(formulas)

def valor_celula(X, N, i, j):
    if 0 <= i <= N and 0 <= j <= N:
        return X[i, j]
    if 0 <= i <= N + 1 and 0 <= j <= N + 1 and (i == N + 1 or j == N + 1):
        return Bool(False)
    raise ValueError(f"Posição inválida: {(i, j)}")

def regra(X, N, i, j):
    if not (1 <= i <= N and 1 <= j <= N):
        raise ValueError("A regra aplica-se apenas a células normais")
    vizinhos = [valor_celula(X, N, i + di, j + dj)
                for di in (-1, 0, 1) for dj in (-1, 0, 1) if (di, dj) != (0, 0)]
    vivos = Plus([Ite(v, Int(1), Int(0)) for v in vizinhos])
    return Or(Equals(vivos, Int(3)), And(X[i, j], Equals(vivos, Int(2))))

def seletor(prox):
```

```
    # O nome depende do destino, garantindo escolhas independentes entre passos.
    return Symbol("q_" + prox[1, 1].symbol_name(), INT)

def trans(atual, prox, N, modo="simultanea"):
    if modo not in ("simultanea", "individual"):
        raise ValueError("Modo inválido")
    formulas = [Iff(prox[i, j], atual[i, j])
                for i in range(N + 1) for j in range(N + 1) if i == 0 or j == 0]
    if modo == "individual":
        q = seletor(prox)
        formulas += [q >= Int(0), q < Int(N * N)]
    for i in range(1, N + 1):
        for j in range(1, N + 1):
            novo = regra(atual, N, i, j)
            if modo == "individual":
                novo = Ite(Equals(q, Int((i - 1) * N + j - 1)), novo, atual[i, j])
            formulas.append(Iff(prox[i, j], novo))
    return And(formulas)
```

2. Testes de correção da FSM

Os testes verificam a inicialização, o tratamento das fronteiras, a rejeição de posições inválidas, a aplicação da regra B3/S23 e a independência das escolhas no modo individual.

O oscilador *blinker* é utilizado para comparar as duas modalidades. Na atualização simultânea, três células vivas alinhadas horizontalmente originam três células alinhadas verticalmente.

Na atualização individual, apenas a célula selecionada aplica a regra, podendo o estado permanecer inalterado.

A dimensão $N = 5$ permite testar a implementação com menor custo. A verificação principal utiliza dimensões superiores a 16.

In [ ]:
```
# Testes SMT de regressão desenvolvidos com assistência do ChatGPT.
def _testes():
    N = 5
    a, b, c = [declare(f"teste_{t}", N) for t in range(3)]
    fr = fronteira_aleatoria(N, 42)
    fixar = lambda X, vivas: And([v if p in vivas else Not(v) for p, v in
X.items()])
    assert len(a) == 36 and len(fr) == 11
    assert valor_celula(a, N, N + 1, 1).is_false()
    assert valor_celula(a, N, 1, N + 1).is_false()
    for pos, erro_esperado in [((-1, 1), ValueError), ((1, 1), KeyError)]:
        try:
            valor_celula(a if erro_esperado is ValueError else {}, N, *pos)
            raise AssertionError("Era esperado rejeitar a posição")
        except erro_esperado:
            pass
    assert is_sat(init(a, N, fr), solver_name="z3")
    assert not is_sat(And(init(a, N, fr), Not(a[1, 1])), solver_name="z3")
```

```
    assert not is_sat(And(init(a, N, fr), a[4, 4]), solver_name="z3")
    for val in (a[0, 0], Not(a[0, 0])):
        assert is_sat(And(init(a, N), val), solver_name="z3")
    for modo in ("simultanea", "individual"):
        assert not is_sat(And(trans(a, b, N, modo), a[0, 0], Not(b[0, 0])),
solver_name="z3")
    horizontal, vertical = {(2, 1), (2, 2), (2, 3)}, {(1, 2), (2, 2), (3, 2)}
    base = fixar(a, horizontal)
    sim, ind = trans(a, b, N, "simultanea"), trans(a, b, N, "individual")
    assert is_sat(And(base, sim, fixar(b, vertical)), solver_name="z3")
    assert not is_sat(And(base, sim, Not(fixar(b, vertical))), solver_name="z3")
    assert is_sat(And(base, ind, fixar(b, {(2, 2), (2, 3)})), solver_name="z3")
    assert not is_sat(And(base, ind, fixar(b, vertical)), solver_name="z3")
    assert is_sat(And(base, ind, fixar(b, horizontal)), solver_name="z3")
    zeros = {p: 0 for p in fr}
    duas = And(init(a, N, zeros), trans(a, b, N, "individual"),
               trans(b, c, N, "individual"), Equals(seletor(b), Int(24)),
               Equals(seletor(c), Int(6)), init(b, N, zeros), Not(c[2, 2]), c[1,
1])
    assert is_sat(duas, solver_name="z3")
    return "**Testes FSM N=5: todos passaram.**"

mo.md(_testes())
```

Testes FSM N=5: todos passaram.

3. Propriedade P1 — estado de erro inalcançável

Um estado de erro corresponde a uma configuração em que todas as células normais estão mortas:

$$E(X) = \bigwedge_{i=1}^{N} \bigwedge_{j=1}^{N} \neg x_{i,j}$$

A propriedade de segurança é:

$$P_1 = G\neg E$$

Bounded Model Checking

Para uma profundidade k, verificamos:

$$I(X_0) \wedge \bigwedge_{t=0}^{k-1} T(X_t, X_{t+1}) \wedge E(X_k)$$

`SAT` com percurso validado demonstra que o estado de erro é alcançável e refuta P1.

`UNSAT` para todas as profundidades até K apenas exclui contraexemplos dentro desse limite.

Indução simples e k-indução

A base exclui estados de erro alcançáveis nas profundidades $0, \ldots, k-1$.

O passo procura uma sequência de k transições na qual P é verdadeira nos primeiros k estados e falsa no último.

O primeiro estado do passo não é sujeito a `init`, embora deva respeitar a fronteira quando esta é fixa.

Um passo indutivo `SAT` pode começar num estado inalcançável, pelo que não constitui necessariamente um contraexemplo real.

In [ ]:
```python
# BMC e validação independente implementados com assistência do ChatGPT.
from pysmt.shortcuts import Solver as SolverSMT

def error(X, N):
    return And([Not(X[i, j]) for i in range(1, N + 1) for j in range(1, N + 1)])

def validar_percurso(estados, N, modo, escolhas, fronteira=None):
    if modo not in ("individual", "simultanea") or len(estados) != len(escolhas) +
1:
        raise ValueError("Percurso inválido")
    posicoes = {(i, j) for i in range(N + 1) for j in range(N + 1)}
    borda = {p for p in posicoes if 0 in p}

    def ler(s, i, j):
        if 0 <= i <= N and 0 <= j <= N:
            return int(s[i, j])
        if 0 <= i <= N + 1 and 0 <= j <= N + 1 and (i == N + 1 or j == N + 1):
            return 0
        raise ValueError(f"Posição inválida: {(i, j)}")

    for s in estados:
        assert set(s) == posicoes and all(type(v) is bool for v in s.values())
        assert all(s[p] == estados[0][p] for p in borda)
        if fronteira is not None:
            assert all(s[p] == bool(fronteira[p]) for p in borda)
    assert all(estados[0][i, j] == (i <= 3 and j <= 3)
               for i in range(1, N + 1) for j in range(1, N + 1))

    for t, (antes, depois) in enumerate(zip(estados, estados[1:])):
        q = escolhas[t]
        assert (type(q) is int and 0 <= q < N * N) if modo == "individual" else q
is None
        for i in range(1, N + 1):
            for j in range(1, N + 1):
                n = sum(ler(antes, i + di, j + dj)
                        for di in (-1, 0, 1) for dj in (-1, 0, 1) if (di, dj) !=
(0, 0))
                novo = (n == 3 or (antes[i, j] and n == 2))
                esperado = (novo if modo == "simultanea" or q == (i - 1) * N + j -
1
                            else antes[i, j])
```

```
                assert depois[i, j] == esperado, ("Transição inválida", t, i, j)
    return True

def validar_traco(estados, N, modo, escolhas, fronteira=None):
    validar_percurso(estados, N, modo, escolhas, fronteira)
    assert not any(estados[-1][i, j]
                   for i in range(1, N + 1) for j in range(1, N + 1))
    return True

def bmc_erro(N, modo, K, fronteira=None):
    if K < 0:
        raise ValueError("K inválido")
    X = [declare(f"bmc_{modo}_{N}_{t}", N) for t in range(K + 1)]
    with SolverSMT(name="z3") as s:
        s.add_assertion(init(X[0], N, fronteira))
        for k in range(K + 1):
            if k:
                s.add_assertion(trans(X[k - 1], X[k], N, modo))
            s.push()
            s.add_assertion(error(X[k], N))
            if s.solve():
                estados = [{p: bool(s.get_py_value(v)) for p, v in st.items()} for
st in X[:k + 1]]
                escolhas = ([int(s.get_py_value(seletor(X[t + 1]))) for t in
range(k)]
                            if modo == "individual" else [None] * k)
                validar_traco(estados, N, modo, escolhas, fronteira)
                return {"k": k, "estados": estados, "escolhas": escolhas,
"validado": True}
            s.pop()
    return None
```

In [ ]:

```
# Obrigações indutivas implementadas com assistência do ChatGPT.
def formula_passo(N, modo, k, fronteira=None, tag="passo"):
    if k < 1:
        raise ValueError("k deve ser positivo")
    X = [declare(f"{tag}_{modo}_{N}_{k}_{t}", N) for t in range(k + 1)]
    formulas = ([Not(error(X[t], N)) for t in range(k)]
                + [trans(X[t], X[t + 1], N, modo) for t in range(k)]
                + [error(X[k], N)])
    if fronteira is not None:
        # Restringe a fronteira sem impor acessibilidade ao estado inicial do
passo.
        formulas += [X[0][p] if v else Not(X[0][p]) for p, v in fronteira.items()]
    return And(formulas), X

def k_inducao(N, modo, k, fronteira=None):
    base = bmc_erro(N, modo, k - 1, fronteira)
    if base is not None:
        return {"base": "SAT", "passo": "não executado", "conclusao": "refutada",
"k_erro": base["k"]}
    formula, X = formula_passo(N, modo, k, fronteira, "kind")
    with SolverSMT(name="z3") as s:
        s.add_assertion(formula)
        if s.solve():
            if fronteira is not None:
```

```
                assert all(bool(s.get_py_value(X[t][p])) == bool(v)
                           for t in range(k + 1) for p, v in fronteira.items())
            vivos = [sum(bool(s.get_py_value(X[t][i, j]))
                         for i in range(1, N + 1) for j in range(1, N + 1))
                     for t in range(k + 1)]
            return {"base": "UNSAT", "passo": "SAT", "conclusao": "inconclusiva",
"vivas": vivos}
    return {"base": "UNSAT", "passo": "UNSAT", "conclusao": "provada"}

def inducao_simples(N, modo, fronteira=None):
    return k_inducao(N, modo, 1, fronteira)
```

In [ ]:
```
botao_p1 = mo.ui.run_button(label="Executar BMC e indução (N=5 e N=17)")
mo.vstack([mo.md("### 3.1. Resultados de segurança e indução"), botao_p1])
```

3.1. Resultados de segurança e indução

Executar BMC e indução (N=5 e N=17)

In [ ]:
```
mo.stop(not botao_p1.value, mo.md("*Experiência não executada nesta sessão.*"))

def _executar_p1():
    with mo.status.spinner(title="A executar BMC e indução…"):
        N = 17
        fr = fronteira_aleatoria(N, 42)
        a, b = declare("reg_n17_a", N), declare("reg_n17_b", N)
        verificacao = [is_sat(And(init(a, N, fr), trans(a, b, N, m)),
solver_name="z3")
                       for m in ("simultanea", "individual")]
        assert all(verificacao) and len(a) == 324 and len(fr) == 35
        controlo = {p: 0 for p in fronteira_aleatoria(3, 42)}
        c = bmc_erro(3, "simultanea", 2, controlo)
        assert c and c["k"] == 2 and c["validado"]
        linhas = ["| Teste | Resultado |", "|:--|:--|",
                  "| FSM N=17, um passo nos dois modos | SAT / SAT |",
                  "| BMC de controlo N=3, K=2 | SAT; percurso validado |"]
        for modo in ("simultanea", "individual"):
            for nome, f in (("fixa42", fronteira_aleatoria(5, 42)), ("simbólica",
None)):
                r = bmc_erro(5, modo, 2, f)
                linhas.append(f"| BMC N=5, {modo}, {nome}, K=2 | {'SAT em
k='+str(r['k']) if r else 'UNSAT até K'} |")
        linhas.append("| BMC N=17, simultânea, fixa42, K=1 | "
                      + ("SAT" if bmc_erro(17, "simultanea", 1, fr) else "UNSAT
até K") + " |")
        for modo in ("simultanea", "individual"):
            r = inducao_simples(5, modo, None)
            assert r["conclusao"] == "inconclusiva"
            linhas.append(f"| Indução N=5, {modo}, simbólica | BASE {r['base']};
PASSO {r['passo']} |")
        for k in (2, 3):
            r = k_inducao(5, "individual", k, None)
            assert r["conclusao"] == "inconclusiva"
            linhas.append(f"| {k}-indução N=5 individual | BASE {r['base']}; PASSO
{r['passo']} |")
```

```
        r = k_inducao(5, "simultanea", 2, None)
        linhas.append(f"| 2-indução N=5 simultânea | {r['conclusao']} |")
        for modo in ("simultanea", "individual"):
            for nome, f in (("fixa42", fr), ("simbólica", None)):
                r = inducao_simples(17, modo, f)
                linhas.append(f"| Indução N=17, {modo}, {nome} | BASE {r['base']};
PASSO {r['passo']}; {r['conclusao']} |")

    return ("**Execução concluída.**\n\n" + "\n".join(linhas))

mo.md(_executar_p1())
```

Execução concluída.

Teste	Resultado
FSM N=17, um passo nos dois modos	SAT / SAT
BMC de controlo N=3, K=2	SAT; percurso validado
BMC N=5, simultanea, fixa42, K=2	UNSAT até K
BMC N=5, simultanea, simbólica, K=2	UNSAT até K
BMC N=5, individual, fixa42, K=2	UNSAT até K
BMC N=5, individual, simbólica, K=2	UNSAT até K
BMC N=17, simultânea, fixa42, K=1	UNSAT até K
Indução N=5, simultanea, simbólica	BASE UNSAT; PASSO SAT
Indução N=5, individual, simbólica	BASE UNSAT; PASSO SAT
2-indução N=5 individual	BASE UNSAT; PASSO SAT
3-indução N=5 individual	BASE UNSAT; PASSO SAT
2-indução N=5 simultânea	inconclusiva
Indução N=17, simultanea, fixa42	BASE UNSAT; PASSO SAT; inconclusiva
Indução N=17, simultanea, simbólica	BASE UNSAT; PASSO SAT; inconclusiva
Indução N=17, individual, fixa42	BASE UNSAT; PASSO SAT; inconclusiva
Indução N=17, individual, simbólica	BASE UNSAT; PASSO SAT; inconclusiva

4. Propriedade P2 — permanência das células

A segunda propriedade exige que cada célula normal esteja viva e morta infinitas vezes ao longo de uma execução:

$$P_2 = \bigwedge_{c \in C} (GF\, x_c \wedge GF\, \neg x_c)$$

Para refutar P2, basta encontrar uma célula que, a partir de determinado instante, permaneça indefinidamente viva ou morta.

Verificação com laços

Procuramos uma sequência:

$$X_0 \to \cdots \to X_k \to X_{k+1} = X_\ell$$

Na parte cíclica, a célula observada conserva o mesmo valor.

É indispensável verificar a relação de transição $T(X_k, X_{k+1})$, além da igualdade de estados.

O estado adicional X_{k+1} permite representar separadamente a transição de fecho, sem reutilizar indevidamente o seletor de outra transição.

Um ciclo válido pode repetir-se indefinidamente, constituindo um contraexemplo à propriedade.

A ausência de ciclos até uma profundidade limitada não demonstra a propriedade temporal global.

In [ ]:
```
# Verificação temporal e validador desenvolvidos com assistência do ChatGPT.
def validar_laco(estados, N, modo, escolhas, inicio_laco, celula, fronteira=None):
    validar_percurso(estados, N, modo, escolhas, fronteira)
    k = len(estados) - 2
    if not (0 <= inicio_laco <= k and 1 <= celula[0] <= N and 1 <= celula[1] <=
N):
        raise ValueError("Ciclo ou célula inválidos")
    assert estados[-1] == estados[inicio_laco], "O laço não fecha"
    assert all(estados[t][celula] == estados[inicio_laco][celula]
               for t in range(inicio_laco, k + 1)), "A célula muda no ciclo"
    return True

def bmc_permanencia(N, modo, K, fronteira=None, celula=None, escolher=None):
    if K < 0 or (escolher is not None and modo != "individual"):
        raise ValueError("Parâmetros inválidos")
    if celula is not None and not (1 <= celula[0] <= N and 1 <= celula[1] <= N):
        raise ValueError("Célula inválida")
    if escolher is not None and not (1 <= escolher[0] <= N and 1 <= escolher[1] <=
N):
        raise ValueError("Escolha inválida")
    for k in range(K + 1):
        X = [declare(f"laco_{modo}_{N}_{k}_{t}", N) for t in range(k + 2)]
        formulas = [init(X[0], N, fronteira)] + [trans(X[t], X[t + 1], N, modo)
                                                 for t in range(k + 1)]
        if escolher is not None:
            indice = (escolher[0] - 1) * N + escolher[1] - 1
            formulas += [Equals(seletor(X[t + 1]), Int(indice)) for t in range(k +
1)]
        with SolverSMT(name="z3") as s:
            s.add_assertion(And(formulas))
            for l in range(k + 1):
```

```
                alvos = [celula] if celula is not None else [
                    (i, j) for i in range(1, N + 1) for j in range(1, N + 1)]
                fecho = And([Iff(X[k + 1][p], X[l][p]) for p in X[l]])
                estavel = Or([Or(And([X[t][p] for t in range(l, k + 1)]),
                                 And([Not(X[t][p]) for t in range(l, k + 1)])) for
p in alvos])
                s.push()
                s.add_assertion(And(fecho, estavel))
                if s.solve():
                    estados = [{p: bool(s.get_py_value(v)) for p, v in st.items()}
for st in X]
                    escolhas = ([int(s.get_py_value(seletor(X[t + 1]))) for t in
range(k + 1)]
                                if modo == "individual" else [None] * (k + 1))
                    alvo = next(p for p in alvos if all(estados[t][p] ==
estados[l][p]
                                                        for t in range(l, k + 1)))
                    validar_laco(estados, N, modo, escolhas, l, alvo, fronteira)
                    return {"k": k, "inicio": l, "celula": alvo, "viva":
estados[l][alvo],
                            "estados": estados, "escolhas": escolhas, "validado":
True}
                s.pop()
    return None
```

In [ ]:
```
botao_p2 = mo.ui.run_button(label="Pesquisar laços P2 (N=5 e N=17)")
mo.vstack([mo.md("### 4.1. Resultados da pesquisa temporal"), botao_p2])
```

4.1. Resultados da pesquisa temporal

Pesquisar laços P2 (N=5 e N=17)

In [ ]:
```
mo.stop(not botao_p2.value, mo.md("*Experiência não executada nesta sessão.*"))

def _executar_p2():
    with mo.status.spinner(title="A pesquisar laços…"):
        linhas = ["| Configuração | Resultado |", "|:--|:--|"]
        for nome, fr in (("fixa42", fronteira_aleatoria(17, 42)), ("simbólica",
None)):
            r = bmc_permanencia(17, "individual", 0, fr, (17, 17), (17, 17))
            assert r and r["validado"] and r["k"] == 0 and r["escolhas"] == [288]
            linhas.append(f"| Individual N=17, {nome}, K=0 | SAT; laço validado em
{r['inicio']} |")
        for N, K in ((5, 1), (17, 0)):
            for nome, fr in (("fixa42", fronteira_aleatoria(N, 42)), ("simbólica",
None)):
                r = bmc_permanencia(N, "simultanea", K, fr)
                linhas.append(f"| Simultânea N={N}, {nome}, K={K} | "
                              + ("SAT, ciclo validado" if r else "UNSAT até K
(limitado)") + " |")

    return ("**Execução concluída.**\n\n" + "\n".join(linhas))

mo.md(_executar_p2())
```

Execução concluída.

Configuração	Resultado
Individual N=17, fixa42, K=0	SAT; laço validado em 0
Individual N=17, simbólica, K=0	SAT; laço validado em 0
Simultânea N=5, fixa42, K=1	UNSAT até K (limitado)
Simultânea N=5, simbólica, K=1	UNSAT até K (limitado)
Simultânea N=17, fixa42, K=0	UNSAT até K (limitado)
Simultânea N=17, simbólica, K=0	UNSAT até K (limitado)

5. Contraexemplos e validação independente

Foram construídas testemunhas concretas para as duas propriedades e para ambas as modalidades de atualização.

P1 — Estado de erro alcançável

Modalidade simultânea

Fronteira inicial viva apenas em $(0, 2)$ e $(2, 0)$.

Evolução do número de células normais vivas:

$$9 \to 3 \to 1 \to 0$$

O estado de erro é alcançado após três transições.

Modalidade individual

Fronteira inicial toda morta.

Sequência de células selecionadas:

$$(1, 2), (2, 1), (1, 1), (2, 3), (2, 2), (3, 1), (3, 2), (3, 3), (1, 3)$$

Cada atualização elimina a célula selecionada. Após nove transições, todas as células normais estão mortas.

P2 — Permanência viva ou morta

Modalidade individual

A seleção repetida de $(17, 17)$ conserva o estado inicial, pois essa célula começa morta e não tem vizinhos vivos.

Temos um auto-laço $X_0 \to X_0$. A célula $(1, 1)$ permanece viva e $(17, 17)$ permanece morta.

Modalidade simultânea

Com a fronteira toda morta, a evolução apresenta:

$$9 \to 6 \to 5 \to 7 \to 7 \to 7$$

Os estados X_4 e X_5 coincidem e a relação $T(X_4, X_5)$ é satisfeita.

O estado X_4 constitui um ponto fixo, no qual $(1, 3)$ permanece viva.

Validação

As testemunhas são representadas por estados concretos e confrontadas com as fórmulas `init`, `trans` e `error`, quando aplicável.

Os estados e as transições são posteriormente verificados por validadores Python independentes do solver.

A existência de uma configuração inicial admissível com contraexemplo é suficiente para refutar a validade universal da propriedade na versão de fronteira simbólica.

Estes resultados não demonstram que todas as fronteiras fixas produzam os mesmos percursos.

In [ ]:
```
# Testemunhas e verificação SMT desenvolvidas com assistência do ChatGPT.
def simular(N, modo, fronteira, escolhas):
    """Calcula os estados sem consultar a FSM pySMT/Z3."""
    atual = {(i, j): (1 <= i <= 3 and 1 <= j <= 3)
             for i in range(N + 1) for j in range(N + 1)}
    atual.update({p: bool(v) for p, v in fronteira.items()})
    estados = [atual]

    def ler(s, i, j):
        return int(s[i, j]) if i <= N and j <= N else 0

    for q in escolhas:
        antes = estados[-1]
        depois = antes.copy()
        posicoes = ([(i, j) for i in range(1, N + 1) for j in range(1, N + 1)]
                    if modo == "simultanea" else [(q // N + 1, q % N + 1)])
        for i, j in posicoes:
            viz = sum(ler(antes, i + di, j + dj)
                      for di in (-1, 0, 1) for dj in (-1, 0, 1) if (di, dj) != (0,
0))
            depois[i, j] = viz == 3 or (antes[i, j] and viz == 2)
        estados.append(depois)
    return estados

def construir_testemunha(N, propriedade, modo):
    if N < 5 or propriedade not in ("P1", "P2") or modo not in ("individual",
"simultanea"):
```

```
        raise ValueError("Parâmetros inválidos")
    fr = {(i, j): 0 for i in range(N + 1) for j in range(N + 1) if i == 0 or j ==
0}
    if propriedade == "P1" and modo == "simultanea":
        fr[(0, 2)] = fr[(2, 0)] = 1
        escolhas = [None] * 3
    elif propriedade == "P1":
        ordem = [(1, 2), (2, 1), (1, 1), (2, 3), (2, 2), (3, 1), (3, 2), (3, 3),
(1, 3)]
        escolhas = [(i - 1) * N + j - 1 for i, j in ordem]
    else:
        escolhas = ([None] * 5 if modo == "simultanea" else [N * N - 1])
    estados = simular(N, modo, fr, escolhas)
    contagens = [sum(s[i, j] for i in range(1, N + 1) for j in range(1, N + 1))
                 for s in estados]
    if propriedade == "P1":
        assert contagens == ([9, 3, 1, 0] if modo == "simultanea" else
list(range(9, -1, -1)))
        validar_traco(estados, N, modo, escolhas, fr)
    else:
        assert contagens == ([9, 6, 5, 7, 7, 7] if modo == "simultanea" else [9,
9])
        validar_laco(estados, N, modo, escolhas, 4 if modo == "simultanea" else 0,
                     (1, 3) if modo == "simultanea" else (N, N), fr)
    return estados, fr, escolhas, contagens

def fixar_estado(X, s):
    return And([X[p] if v else Not(X[p]) for p, v in s.items()])

def confirmar_testemunha(N, propriedade, modo, simbolica=False):
    estados_py, fr, escolhas, contagens = construir_testemunha(N, propriedade,
modo)
    X = [declare(f"test_{propriedade}_{modo}_{N}_{simbolica}_{t}", N)
         for t in range(len(estados_py))]
    formulas = [init(X[0], N, None if simbolica else fr)]
    formulas += [trans(X[t], X[t + 1], N, modo) for t in range(len(escolhas))]
    if modo == "individual":
        formulas += [Equals(seletor(X[t + 1]), Int(q)) for t, q in
enumerate(escolhas)]
    limite = len(X) if propriedade == "P1" else len(X) - 1
    formulas += [fixar_estado(X[t], estados_py[t]) for t in range(limite)]
    if propriedade == "P1":
        formulas.append(error(X[-1], N))
    else:
        # Verifica o fecho em todas as células, incluindo a fronteira.
        inicio = 4 if modo == "simultanea" else 0
        formulas.append(And([Iff(X[-1][p], X[inicio][p]) for p in X[0]]))
    with SolverSMT(name="z3") as s:
        s.add_assertion(And(formulas))
        if not s.solve():
            raise AssertionError("Testemunha incompatível com a FSM SMT")
        estados_smt = [{p: bool(s.get_py_value(v)) for p, v in X[t].items()}
                       for t in range(len(X))]
        if modo == "individual":
            assert [int(s.get_py_value(seletor(X[t + 1]))) for t in
range(len(escolhas))] == escolhas
```

```
    assert estados_smt == estados_py
    if propriedade == "P1":
        validar_traco(estados_smt, N, modo, escolhas, fr)
    else:
        validar_laco(estados_smt, N, modo, escolhas,
                     4 if modo == "simultanea" else 0,
                     (1, 3) if modo == "simultanea" else (N, N), fr)
    return {"N": N, "propriedade": propriedade, "modo": modo, "estado": "SAT",
            "contagens": contagens, "validado": True}
```

In [ ]:
```
botao_testemunhas = mo.ui.run_button(label="Confirmar testemunhas P1 e P2 com Z3")
mo.vstack([mo.md("### 5.1. Resultados da confirmação SMT"), botao_testemunhas])
```

5.1. Resultados da confirmação SMT

Confirmar testemunhas P1 e P2 com Z3

In [ ]:
```
mo.stop(not botao_testemunhas.value, mo.md("*Experiência não executada nesta sessão.*"))

def _executar_testemunhas():
    with mo.status.spinner(title="A validar os contraexemplos…"):
        linhas = ["| P | N | Modalidade | Fronteira | Resultado | Células vivas |", "|:--|--:|:--|:--|:--|:--|"]
        for p, modo, N, simb in ([("P1", m, n, False) for n in (5, 17) for m in ("simultanea", "individual")]
                                 + [("P1", m, 17, True) for m in ("simultanea", "individual")]
                                 + [("P2", m, n, False) for n in (5, 17) for m in ("simultanea", "individual")]
                                 + [("P2", "simultanea", 17, True)]):
            r = confirmar_testemunha(N, p, modo, simb)
            assert r["validado"]
            linhas.append(f"| {p} | {N} | {modo} | {'simbólica, testemunha fixada' if simb else 'concreta'} "
                          f"| SAT + validador OK | {r['contagens']} |")

    return ("**Execução concluída.**\n\n" + "\n".join(linhas))

mo.md(_executar_testemunhas())
```

Execução concluída.

P	N	Modalidade	Fronteira	Resultado	Células vivas
P1	5	simultanea	concreta	SAT + validador OK	[9, 3, 1, 0]
P1	5	individual	concreta	SAT + validador OK	[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
P1	17	simultanea	concreta	SAT + validador OK	[9, 3, 1, 0]
P1	17	individual	concreta	SAT + validador OK	[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
P1	17	simultanea	simbólica, testemunha fixada	SAT + validador OK	[9, 3, 1, 0]
P1	17	individual	simbólica, testemunha fixada	SAT + validador OK	[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
P2	5	simultanea	concreta	SAT + validador OK	[9, 6, 5, 7, 7, 7]
P2	5	individual	concreta	SAT + validador OK	[9, 9]
P2	17	simultanea	concreta	SAT + validador OK	[9, 6, 5, 7, 7, 7]
P2	17	individual	concreta	SAT + validador OK	[9, 9]
P2	17	simultanea	simbólica, testemunha fixada	SAT + validador OK	[9, 6, 5, 7, 7, 7]

6. Desempenho e escalabilidade

As experiências foram realizadas com três repetições por modalidade, utilizando $N = 17$ e $N = 21$.

Ambiente experimental

- Sistema operativo: Windows 11.
- Python: 3.14.7.
- pySMT: 0.9.6.
- z3-solver: 5.3.2.0.
- Marimo: 0.25.0.
- Solver: Z3, com limite de 5000 ms por consulta.

A fronteira foi fixada com semente 42, exceto nas experiências de laços, que utilizaram uma fronteira toda morta.

Metodologia

Para cada repetição, mediram-se três tempos com `time.perf_counter()`:

- **Construção:** declaração das variáveis e construção da fórmula pySMT.
- **Solver:** criação, introdução da fórmula, resolução e libertação dos recursos.
- **Total:** duração completa da experiência.

O limite de tempo do Z3 aplica-se ao solver e não limita a construção das fórmulas.

As consultas SMT não apresentaram erros nem timeouts nas experiências registadas.

Fórmulas medidas

FSM: satisfatibilidade de uma transição a partir do estado inicial.

BMC: erro exatamente no estado X_2, após duas transições. Não corresponde a uma pesquisa de todas as profundidades até 2.

Indução simples: apenas a obrigação do passo indutivo, sem a base.

k-indução: apenas o passo indutivo para $k = 2$, sem os casos de base.

Laços: confirmação da consistência de uma testemunha conhecida.

Na simultânea, a testemunha utiliza cinco transições até um ponto fixo; na individual, utiliza um auto-laço de uma transição.

Os testes de laços utilizam fronteira toda morta independentemente da opção selecionada no controlo do benchmark.

As medições individuais e as respetivas medianas são apresentadas nas tabelas seguintes.

In [ ]:
```
def _render_medicoes():
    from statistics import median
    from html import escape

    # Resultados experimentais observados localmente.
    # Organização e apresentação assistidas pelo ChatGPT.
    MEDICOES = """
    FSM|17|S|SAT|10.0,42.3,52.3;10.1,37.9,48.0;10.8,37.0,47.8
    FSM|17|I|SAT|11.1,68.6,79.7;10.4,59.6,70.1;15.7,57.0,72.6
    FSM|21|S|SAT|72.8,52.3,125.1;57.5,60.8,118.3;65.0,63.0,128.0
    FSM|21|I|SAT|55.6,104.6,160.1;56.6,98.3,154.9;52.0,94.2,146.1
    BMC k=2|17|S|UNSAT|61.2,72.2,133.4;64.7,69.3,134.0;69.9,68.7,138.6
    BMC k=2|17|I|UNSAT|74.9,94.9,169.8;73.8,90.8,164.6;73.7,85.3,158.9
    BMC k=2|21|S|UNSAT|129.7,139.2,268.9;125.1,124.2,249.3;156.2,116.8,272.9
    BMC k=2|21|I|UNSAT|422.7,152.8,575.5;123.9,179.7,303.6;109.4,146.4,255.8
    Ind. simples|17|S|SAT|33.5,445.6,479.1;38.0,517.4,555.4;32.5,438.8,471.3
    Ind. simples|17|I|SAT|42.5,549.9,592.4;46.6,547.3,593.9;41.2,556.2,597.4
    Ind. simples|21|S|SAT|71.0,693.6,764.6;62.4,652.5,714.8;346.8,837.5,1184.3
```

```
Ind. simples|21|I|SAT|61.7,774.9,836.7;61.9,792.4,854.3;65.2,1037.3,1102.9
k-ind. k=2|17|S|SAT|69.7,1293.8,1363.5;68.3,1325.5,1393.9;65.2,1205.8,1271.0
k-ind. k=2|17|I|SAT|74.4,1504.9,1579.3;76.2,1408.0,1484.2;74.4,1480.5,1554.9
Laço P2|17|S|SAT|159.7,185.5,345.2;164.3,208.2,372.5;150.5,178.0,328.4
Laço P2|17|I|SAT|38.0,61.5,99.5;55.0,66.5,121.5;425.2,77.8,503.0
"""

# Extrair os 48 registos individuais.
dados = []

for linha in MEDICOES.strip().splitlines():
    metodo, N, modo, estado, medicoes = (
        linha.strip().split("|")
    )

    for repeticao, triplo in enumerate(
        medicoes.split(";"), 1
    ):
        construcao, solver, total = [
            float(x) for x in triplo.split(",")
        ]

        dados.append((
            metodo, int(N), modo, repeticao,
            construcao, solver, total, estado
        ))

assert len(dados) == 48

# Preparar a tabela das medições individuais.
cabecalho_bruto = [
    "Método", "N", "Modo", "Rep.",
    "Construção", "Z3", "Total", "Estado"
]

linhas_bruto = [
    [
        metodo, N, modo, rep,
        f"{c:.1f}", f"{z:.1f}",
        f"{total:.1f}", estado
    ]
    for metodo, N, modo, rep, c, z, total, estado
    in dados
]

# Calcular as medianas separadamente por métrica.
grupos = list(dict.fromkeys(
    (r[0], r[1], r[2], r[7]) for r in dados
))

assert len(grupos) == 16

cabecalho_resumo = [
    "Método", "N", "Modo",
    "Construção med.", "Z3 med.",
    "Total med.", "Estado"
```

```
    ]

    linhas_resumo = []

    for metodo, N, modo, estado in grupos:
        valores = [
            r for r in dados
            if (r[0], r[1], r[2], r[7])
            == (metodo, N, modo, estado)
        ]

        linhas_resumo.append([
            metodo,
            N,
            modo,
            f"{median(r[4] for r in valores):.1f}",
            f"{median(r[5] for r in valores):.1f}",
            f"{median(r[6] for r in valores):.1f}",
            estado
        ])

    # Gerar tabelas HTML para apresentação e exportação.
    def tabela_html(cabecalho, linhas):
        estilo_tabela = (
            "border-collapse:collapse;"
            "width:100%;font-size:0.85em;"
        )
        estilo_celula = (
            "padding:5px 8px;"
            "border-bottom:1px solid #ddd;"
            "text-align:left;"
        )

        html = (
            f'<table style="{estilo_tabela}">'
            "<thead><tr>"
        )

        for coluna in cabecalho:
            html += (
                f'<th style="{estilo_celula}">'
                f'{escape(str(coluna))}</th>'
            )

        html += "</tr></thead><tbody>"

        for linha in linhas:
            html += "<tr>"

            for valor in linha:
                html += (
                    f'<td style="{estilo_celula}">'
                    f'{escape(str(valor))}</td>'
                )

            html += "</tr>"
```

```
        html += "</tbody></table>"

        return mo.Html(html)

    return mo.vstack([
        mo.md("### 6.1. Medições individuais (ms)"),
        tabela_html(cabecalho_bruto, linhas_bruto),
        mo.md("### 6.2. Medianas (ms)"),
        tabela_html(cabecalho_resumo, linhas_resumo),
        mo.md(
            "*S = simultânea; I = individual. "
            "As medianas das parcelas são calculadas "
            "independentemente da mediana total.*"
        )
    ])

_render_medicoes()
```

6.1. Medições individuais (ms)

Método	N	Modo	Rep.	Construção	Z3	Total	Estado
FSM	17	S	1	10.0	42.3	52.3	SAT
FSM	17	S	2	10.1	37.9	48.0	SAT
FSM	17	S	3	10.8	37.0	47.8	SAT
FSM	17	I	1	11.1	68.6	79.7	SAT
FSM	17	I	2	10.4	59.6	70.1	SAT
FSM	17	I	3	15.7	57.0	72.6	SAT
FSM	21	S	1	72.8	52.3	125.1	SAT
FSM	21	S	2	57.5	60.8	118.3	SAT
FSM	21	S	3	65.0	63.0	128.0	SAT
FSM	21	I	1	55.6	104.6	160.1	SAT
FSM	21	I	2	56.6	98.3	154.9	SAT
FSM	21	I	3	52.0	94.2	146.1	SAT
BMC k=2	17	S	1	61.2	72.2	133.4	UNSAT
BMC k=2	17	S	2	64.7	69.3	134.0	UNSAT
BMC k=2	17	S	3	69.9	68.7	138.6	UNSAT
BMC k=2	17	I	1	74.9	94.9	169.8	UNSAT
BMC k=2	17	I	2	73.8	90.8	164.6	UNSAT
BMC k=2	17	I	3	73.7	85.3	158.9	UNSAT
BMC k=2	21	S	1	129.7	139.2	268.9	UNSAT
BMC k=2	21	S	2	125.1	124.2	249.3	UNSAT
BMC k=2	21	S	3	156.2	116.8	272.9	UNSAT
BMC k=2	21	I	1	422.7	152.8	575.5	UNSAT
BMC k=2	21	I	2	123.9	179.7	303.6	UNSAT
BMC k=2	21	I	3	109.4	146.4	255.8	UNSAT
Ind. simples	17	S	1	33.5	445.6	479.1	SAT
Ind. simples	17	S	2	38.0	517.4	555.4	SAT
Ind. simples	17	S	3	32.5	438.8	471.3	SAT
Ind. simples	17	I	1	42.5	549.9	592.4	SAT
Ind. simples	17	I	2	46.6	547.3	593.9	SAT

Método	N	Modo	Rep.	Construção	Z3	Total	Estado
Ind. simples	17	I	3	41.2	556.2	597.4	SAT
Ind. simples	21	S	1	71.0	693.6	764.6	SAT
Ind. simples	21	S	2	62.4	652.5	714.8	SAT
Ind. simples	21	S	3	346.8	837.5	1184.3	SAT
Ind. simples	21	I	1	61.7	774.9	836.7	SAT
Ind. simples	21	I	2	61.9	792.4	854.3	SAT
Ind. simples	21	I	3	65.2	1037.3	1102.9	SAT
k-ind. k=2	17	S	1	69.7	1293.8	1363.5	SAT
k-ind. k=2	17	S	2	68.3	1325.5	1393.9	SAT
k-ind. k=2	17	S	3	65.2	1205.8	1271.0	SAT
k-ind. k=2	17	I	1	74.4	1504.9	1579.3	SAT
k-ind. k=2	17	I	2	76.2	1408.0	1484.2	SAT
k-ind. k=2	17	I	3	74.4	1480.5	1554.9	SAT
Laço P2	17	S	1	159.7	185.5	345.2	SAT
Laço P2	17	S	2	164.3	208.2	372.5	SAT
Laço P2	17	S	3	150.5	178.0	328.4	SAT
Laço P2	17	I	1	38.0	61.5	99.5	SAT
Laço P2	17	I	2	55.0	66.5	121.5	SAT
Laço P2	17	I	3	425.2	77.8	503.0	SAT

6.2. Medianas (ms)

Método	N	Modo	Construção med.	Z3 med.	Total med.	Estado
FSM	17	S	10.1	37.9	48.0	SAT
FSM	17	I	11.1	59.6	72.6	SAT
FSM	21	S	65.0	60.8	125.1	SAT
FSM	21	I	55.6	98.3	154.9	SAT
BMC k=2	17	S	64.7	69.3	134.0	UNSAT
BMC k=2	17	I	73.8	90.8	164.6	UNSAT
BMC k=2	21	S	129.7	124.2	268.9	UNSAT

Método	N	Modo	Construção med.	Z3 med.	Total med.	Estado
BMC k=2	21	I	123.9	152.8	303.6	UNSAT
Ind. simples	17	S	33.5	445.6	479.1	SAT
Ind. simples	17	I	42.5	549.9	593.9	SAT
Ind. simples	21	S	71.0	693.6	764.6	SAT
Ind. simples	21	I	61.9	792.4	854.3	SAT
k-ind. k=2	17	S	68.3	1293.8	1363.5	SAT
k-ind. k=2	17	I	74.4	1480.5	1554.9	SAT
Laço P2	17	S	159.7	185.5	345.2	SAT
Laço P2	17	I	55.0	66.5	121.5	SAT

S = simultânea; I = individual. As medianas das parcelas são calculadas independentemente da mediana total.

6.3. Análise dos resultados

A passagem de $N = 17$ para $N = 21$ aumenta o número de variáveis booleanas por estado de 324 para 484, aproximadamente 49,4%.

Observou-se um aumento das medianas dos tempos totais nos três métodos medidos em ambas as dimensões: FSM, BMC e indução simples.

Estes resultados evidenciam um aumento do custo computacional nas instâncias consideradas, mas não permitem inferir uma lei geral de crescimento temporal.

Nas fórmulas comparáveis, as medianas da modalidade individual foram superiores às da simultânea.

Esta observação não demonstra que a modalidade individual seja sempre computacionalmente mais exigente.

O benchmark de laços envolve percursos de comprimentos diferentes, pelo que os seus tempos não representam tarefas SMT equivalentes.

6.4. Variabilidade

No laço individual com $N = 17$, os tempos totais observados foram:

Repetição	Tempo total
1	99,5 ms
2	121,5 ms

Repetição	Tempo total
3	503,0 ms
Mediana	121,5 ms

A terceira repetição apresentou um aumento considerável, sobretudo no tempo de construção.

Também se observaram valores de construção superiores aos restantes em BMC $N = 21$ individual e indução simples $N = 21$ simultânea.

A causa destas diferenças não foi determinada. A apresentação dos valores individuais impede que esta variabilidade seja ocultada pela mediana.

Na terceira repetição da indução simples $N = 21$ individual, a soma dos tempos parciais é 1102,5 ms, enquanto o total registado foi 1102,9 ms.

A diferença de 0,4 ms está documentada, mantendo-se os valores observados originalmente.

6.5. Interpretação dos resultados SMT

- **FSM SAT:** existe pelo menos uma transição permitida.
- **BMC UNSAT:** o estado de erro não ocorre exatamente em X_2 na configuração de fronteira analisada.
- **Passo indutivo SAT:** existe uma sequência que viola a obrigação indutiva, podendo começar num estado inalcançável.
- **Laço SAT:** a testemunha considerada satisfaz as restrições SMT, incluindo a relação de fecho.

A confirmação da validade de um contraexemplo exige ainda a verificação independente das suas transições.

Os tempos não incluem a extração e a validação dos modelos.

As três repetições por cenário permitem uma avaliação exploratória, mas são insuficientes para caracterizar estatisticamente a distribuição dos tempos de execução.

In [ ]:
```
# Benchmark desenvolvido com assistência do ChatGPT, utilizando a mesma FSM.
def formula_experiencia(N, modo, metodo, fronteira_tipo, repeticao):
    if N < 17:
        raise ValueError("N deve ser >= 17 no benchmark")
    fr = ({p: 0 for p in fronteira_aleatoria(N, 42)} if metodo == "laco" else
          fronteira_aleatoria(N, 42) if fronteira_tipo == "fixa42" else None)
    if fr is None and fronteira_tipo != "simbolica":
        raise ValueError("Fronteira inválida")

    passos = {"fsm": 1, "bmc": 2, "ind1": 1, "kind2": 2,
```

```
                "laco": 1 if modo == "individual" else 5}[metodo]

    if metodo in ("ind1", "kind2"):
        formula, _ = formula_passo(N, modo, passos, fr, f"med_{repeticao}")
        return formula, fronteira_tipo, passos

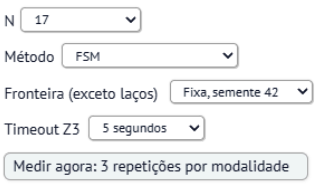
    X = [declare(f"med_{metodo}_{modo}_{N}_{repeticao}_{t}", N) for t in
range(passos + 1)]
    if metodo == "fsm":
        formula = And(init(X[0], N, fr), trans(X[0], X[1], N, modo))
    elif metodo == "bmc":
        formula = And([init(X[0], N, fr), trans(X[0], X[1], N, modo),
                       trans(X[1], X[2], N, modo), error(X[2], N)])
    else:
        estados, _, _, _ = construir_testemunha(N, "P2", modo)
        inicio = 0 if modo == "individual" else 4
        formulas = [init(X[0], N, fr)] + [trans(X[t], X[t + 1], N, modo)
                                          for t in range(passos)]
        if modo == "individual":
            formulas.append(Equals(seletor(X[1]), Int(N * N - 1)))
        else:
            formulas += [fixar_estado(X[t], estados[t]) for t in range(passos)]
        formulas.append(And([Iff(X[-1][p], X[inicio][p]) for p in X[0]]))
        formula = And(formulas)

    return formula, "zeros" if metodo == "laco" else fronteira_tipo, passos
```

In [ ]:
```
n_exp = mo.ui.dropdown(options={"17": 17, "21": 21, "25 (opcional)": 25},
value="17", label="N")
met_exp = mo.ui.dropdown(options={"FSM": "fsm", "BMC exato k=2": "bmc",
                                  "Passo indução simples": "ind1", "Passo k-
indução k=2": "kind2",
                                  "Laço conhecido P2": "laco"}, value="FSM",
label="Método")
fr_exp = mo.ui.dropdown(options={"Fixa, semente 42": "fixa42", "Simbólica":
"simbolica"},
                        value="Fixa, semente 42", label="Fronteira (exceto
laços)")
tempo_exp = mo.ui.dropdown(options={"2 segundos": 2000, "5 segundos": 5000, "10
segundos": 10000},
                           value="5 segundos", label="Timeout Z3")
botao_medir = mo.ui.run_button(label="Medir agora: 3 repetições por modalidade")

mo.vstack([mo.md("### 6.6. Execução de experiências de desempenho"),
           n_exp, met_exp, fr_exp, tempo_exp, botao_medir])
```

6.6. Execução de experiências de desempenho



In [ ]:

```
mo.stop(not botao_medir.value, mo.md("*Nenhuma experiência de desempenho executada
nesta sessão.*"))

def _executar_benchmark():
    from time import perf_counter as _relogio
    from statistics import median as _mediana
    from pysmt.exceptions import SolverReturnedUnknownResultError as _Unknown

    with mo.status.spinner(title="A medir seis execuções…"):
        linhas = []
        for modo in ("simultanea", "individual"):
            for rep in range(1, 4):
                t0 = _relogio()
                f, fr_real, passos = formula_experiencia(n_exp.value, modo,
met_exp.value, fr_exp.value, rep)
                t1 = _relogio()
                estado = "UNKNOWN"
                try:
                    with SolverSMT(name="z3", solver_options={"timeout":
tempo_exp.value}) as s:
                        s.add_assertion(f)
                        estado = "SAT" if s.solve() else "UNSAT"
                except _Unknown:
                    pass
                t2 = _relogio()
                linhas.append((modo, rep, fr_real, passos, estado,
                               round(1000 * (t1 - t0), 1), round(1000 * (t2 - t1),
1),
                               round(1000 * (t2 - t0), 1)))

        tabela = ["| Modo | Rep. | Fronteira | Passos | Estado | Construção (ms) |
Solver (ms) | Total (ms) |",
                  "|:--|--:|:--|--:|:--:|--:|--:|--:|"]
        for modo, rep, fr, passos, estado, c, z, total in linhas:
            tabela.append(f"| {modo} | {rep} | {fr} | {passos} | {estado} |
{c:.1f} | {z:.1f} | {total:.1f} |")
        for modo in ("simultanea", "individual"):
            vals = [r[7] for r in linhas if r[0] == modo]
            tabela.append(f"| **Mediana {modo}** | | | | | | **
{_mediana(vals):.1f}** |")

    return ("**Experiência concluída.**\n\n" + "\n".join(tabela))

mo.md(_executar_benchmark())
```

Experiência concluída.

Modo	Rep.	Fronteira	Passos	Estado	Construção (ms)	Solver (ms)	Total (ms)
simultanea	1	fixa42	1	SAT	311.8	41.1	352.9
simultanea	2	fixa42	1	SAT	34.1	39.5	73.6
simultanea	3	fixa42	1	SAT	40.8	41.2	82.0
individual	1	fixa42	1	SAT	37.8	99.6	137.4
individual	2	fixa42	1	SAT	48.7	99.2	147.9
individual	3	fixa42	1	SAT	44.2	97.6	141.8
Mediana simultanea							**82.0**
Mediana individual							**141.8**

7. Conclusões e limitações

Primeira propriedade

A propriedade de segurança é refutada nas duas modalidades através de percursos que alcançam o estado de erro.

Na modalidade simultânea existe uma testemunha com três transições; na modalidade individual, uma testemunha com nove transições.

Segunda propriedade

A propriedade de alternância entre estados vivos e mortos é refutada por comportamentos cíclicos.

Na modalidade individual existe um auto-laço que conserva o estado inicial.

Na modalidade simultânea existe um ponto fixo alcançável com a fronteira toda morta.

Âmbito das conclusões

As configurações de fronteira utilizadas nas testemunhas são permitidas pelo modelo simbólico.

A existência de uma única execução válida que viola uma propriedade universal é suficiente para a refutar.

Os resultados não demonstram que todas as configurações fixas da fronteira tenham o mesmo comportamento.

Limitações

- A atualização individual pode conservar o valor da célula selecionada.
- Não foi imposta justiça na seleção das células.
- A fronteira simbólica não define probabilidades.
- BMC com profundidade limitada não constitui uma prova de segurança global.
- Um passo indutivo SAT pode envolver estados que não são alcançáveis a partir de `init`.
- A análise experimental considera duas dimensões principais e três repetições por cenário.
- O limite temporal do solver não abrange a construção das fórmulas.
- Os benchmarks de laços utilizam percursos de comprimentos diferentes.

8. Reprodutibilidade

O notebook foi desenvolvido em Python com pySMT, Z3 e Marimo.

Ambiente de referência:

- Windows 11.
- Python 3.14.7.
- pySMT 0.9.6.
- z3-solver 5.3.2.0.
- Marimo 0.25.0.

O ambiente Conda utilizado designa-se `logica`.

O notebook pode ser aberto através do comando:

```
marimo edit nome_do_ficheiro.py
```

Os testes básicos são executados automaticamente. As verificações mais exigentes são executadas através dos controlos apresentados nas secções correspondentes.

As experiências de desempenho permitem selecionar dimensão, método, fronteira e limite temporal.

Cada execução de desempenho realiza três repetições por modalidade e apresenta os tempos individuais, estados do solver e medianas.

Os resultados experimentais originalmente observados encontram-se documentados na secção 6 e podem ser consultados sem repetir as medições.

9. Utilização de ferramentas LLM

O ChatGPT foi utilizado como colaborador técnico no processo de modelação, implementação, verificação e análise do exercício.

As contribuições encontram-se identificadas nos comentários do código e resumidas abaixo.

Componente	Contribuição do ChatGPT	Responsabilidade dos alunos
FSM e regra B3/S23	Apoio à formulação e implementação pySMT	Análise dos requisitos e compreensão do modelo
Modalidades de atualização	Discussão e implementação da interpretação A	Escolha e justificação da interpretação
Fronteiras fixa e simbólica	Proposta e implementação das alternativas	Avaliação do alcance dos resultados
BMC, indução e k-indução	Formulação e implementação dos métodos	Execução, interpretação e análise das limitações
BMC com laços	Modelação e implementação da verificação temporal	Análise dos comportamentos infinitos
Testemunhas P1 e P2	Propostas, simulações e confirmação assistidas	Execução local, análise e validação
Validadores independentes	Implementação Python assistida	Testes e verificação dos resultados
Desempenho	Construção do benchmark e organização dos dados	Execução das 48 medições e análise crítica
Documentação	Apoio à organização e revisão	Revisão do conteúdo e defesa das decisões

Diálogo principal de desenvolvimento: https://chatgpt.com/share/6ae37072-8090-83eb-a790-08c56a03e8d1

10. Referências

- Enunciado do TP2 — Exercício 1.
- Regras gerais dos trabalhos práticos de Lógica Computacional, 2026/2027.
- Documento sobre utilização de ferramentas LLM.
- Materiais da disciplina: capítulos 3 e 4; fichas práticas 7, 8 e 9.
- Documentação oficial do pySMT.
- Documentação oficial do Z3.
- Documentação oficial do Marimo.